

HireIQ

AI Hiring Intelligence Platform

Security & Access Control Document (SACD)

Document Type	Security & Access Control Document (SACD)
Product	HireIQ — AI Hiring Intelligence Platform
Version	v1.0.0
Parent Documents	HireIQ PRD v1.0 HireIQ TAD v1.0
Status	DRAFT — Security Review Required Before Production
Author	Security Engineering Team
Date	June 14, 2026
Classification	CONFIDENTIAL — Restricted Distribution

SECTION 1

Security Overview & Objectives

1. Security Overview & Objectives

This document defines the complete security architecture, threat model, access control policies, data protection measures, compliance requirements, and incident response procedures for the HireIQ AI Hiring Intelligence Platform. It is a mandatory reference for all engineers, DevOps personnel, and auditors working on the platform.

CRITICAL NOTICE

All engineers must read and acknowledge this document before being granted access to any production environment. No exceptions. Security violations are grounds for immediate access revocation.

1.1 Security Objectives

Confidentiality	Ensure that user data (resumes, personal profiles, application history) is accessible only to authorized parties. No data leakage between users or roles.
Integrity	Ensure that all data stored and transmitted cannot be tampered with by unauthorized parties. AI scores and application statuses must be authentic.
Availability	Maintain 99.9% uptime SLA. Security controls must not impede legitimate users. Rate limiting and DDoS protection must not block valid traffic.
Non-Repudiation	All sensitive actions (job creation, status updates, resume uploads) must be logged with actor identity, timestamp, and IP address.
Least Privilege	Every system component, user, and API key must have only the minimum permissions required to perform its function.

1.2 Security Compliance Targets

Standard	Requirement Level	Notes
OWASP Top 10 (2021)	Full compliance required	All 10 categories must pass automated and manual testing before launch
GDPR (EU)	Full compliance required	Right to erasure, data portability, consent management, breach notification

SOC 2 Type I	Target for Month 6	Trust Service Criteria: Security, Availability, Confidentiality
HTTPS / TLS 1.3	Enforced from day one	All traffic encrypted in transit. TLS 1.0 and 1.1 disabled.
NIST CSF	Guidelines followed	Identify, Protect, Detect, Respond, Recover framework adopted

SECTION 2

Threat Model (STRIDE)

2. Threat Model — STRIDE Analysis

We apply the STRIDE threat modeling framework to identify and categorize all threats to the HireIQ platform. STRIDE stands for Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.

2.1 STRIDE Threat Register

Threat (STRIDE)	Description	Risk	Mitigation
S — Spoofing	Attacker impersonates a legitimate user by stealing or forging JWT tokens	High	JWT short expiry (24h), refresh token rotation, httpOnly cookies, token binding in v1.5
S — Spoofing	Attacker creates a recruiter account to harvest job seeker data	Medium	Email verification (v1.1), rate limiting on registration, account flagging system
T — Tampering	Attacker modifies application status in transit (e.g. change rejected to shortlisted)	High	HTTPS TLS 1.3 in transit, JWT signature verification, server-side status validation
T — Tampering	SQL injection to modify database records directly	High	SQLAlchemy ORM parameterized queries, no raw SQL with user input, DB-level constraints
T — Tampering	AI score manipulation by sending crafted resume inputs to inflate scores	Medium	Input length limits, content sanitization, AI endpoint rate limiting per user
R — Repudiation	Recruiter denies changing application status to rejected	Medium	Immutable audit log of all status changes with actor ID, timestamp, IP
R — Repudiation	Job seeker denies uploading a malicious file	Low	S3 upload audit trail, pre-signed URL tied to authenticated user ID
I — Info Disclosure	JWT secret key exposed via GitHub commit or logs	Critical	Secrets in AWS Secrets Manager, gitignore enforced, secret scanning in CI pipeline
I — Info Disclosure	Cross-user data leakage — user A sees user B resume	High	Row-level ownership checks on all data queries, no wildcard DB queries

I — Info Disclosure	S3 bucket misconfiguration exposes all resumes publicly	Critical	S3 public access block enabled, bucket policy denies all public access, ACL disabled
I — Info Disclosure	Verbose error messages expose stack traces to attackers	Medium	Generic error messages in production, full traces only in structured logs (not API responses)
D — Denial of Service	Auth endpoint flooded to lock out legitimate users	High	Redis rate limiting: 100 req/min per IP on /auth/*, progressive backoff, CAPTCHA in v1.1
D — Denial of Service	Large file upload to exhaust server memory	Medium	5MB file size limit enforced at API level before S3 upload, multipart streaming upload
D — Denial of Service	AI endpoint abused to drain GPU/CPU resources	Medium	AI endpoint rate limit per user (10 req/min), async processing queue, resource timeout
E — Elevation of Privilege	Job seeker accesses recruiter-only endpoints	High	RBAC middleware on all routes, role extracted from JWT (not from request body)
E — Elevation of Privilege	Recruiter modifies another recruiter jobs or applications	High	Ownership check: recruiter_id on job must match authenticated user ID
E — Elevation of Privilege	Attacker accesses admin panel without admin role	Critical	Admin role flag separate from recruiter/job_seeker, admin routes behind additional 2FA (v1.1)

2.2 Risk Matrix

Risk Level	Threats in this Category	Required Action
Critical	JWT secret exposure, S3 public access, admin privilege escalation	Immediate fix required — blocks production release
High	Token spoofing, SQL injection, cross-user data leakage, RBAC bypass	Must be fixed before production launch — no exceptions
Medium	AI score manipulation, DDoS on AI endpoints, repudiation attacks	Must be fixed within 30 days of launch
Low	Verbose error messages in edge cases, minor info disclosure	Fix in next sprint — acceptable for soft launch with monitoring

SECTION 3

Authentication Architecture

3. Authentication Architecture

3.1 JWT Authentication Flow

HireIQ uses a dual-token authentication strategy: a short-lived Access Token for API authorization and a long-lived Refresh Token for session continuity.

REGISTRATION FLOW:

1. User POST /auth/register { name, email, password, role }
2. Server: validate input via Pydantic schema (email format, password strength)
3. Server: check users table – return 409 if email already exists
4. Server: hash password with bcrypt (cost=12) – ~250ms intentional delay
5. Server: INSERT into users table
6. Server: generate Access Token (JWT, 24h) + Refresh Token (UUID, 30d)
7. Server: store refresh_token hash in Redis with key refresh:{user_id}
8. Server: return { access_token, user } in body; refresh_token in httpOnly cookie

LOGIN FLOW:

1. User POST /auth/login { email, password }
2. Server: fetch user by email (indexed lookup)
3. Server: bcrypt.verify(input_password, stored_hash) – returns True/False
4. Server: if False → return 401 (generic message, no info about which field failed)
5. Server: generate new Access Token + Refresh Token pair
6. Server: update Redis refresh token store
7. Server: return { access_token, user }; refresh_token in httpOnly cookie

TOKEN REFRESH FLOW:

1. Axios interceptor catches 401 response on any API call
2. Frontend calls POST /auth/refresh (cookie sent automatically by browser)
3. Server: read refresh_token from httpOnly cookie
4. Server: verify token exists in Redis AND is not expired
5. Server: delete old refresh token from Redis (rotation – one-time use)
6. Server: issue new Access Token + new Refresh Token
7. Server: return new access_token; set new refresh_token cookie
8. Axios retries the original failed request with new access_token

LOGOUT FLOW:

1. User POST /auth/logout { } (access_token in header, refresh_token in cookie)
2. Server: delete refresh:{user_id} key from Redis
3. Server: return 200 with Set-Cookie: refresh_token=; Max-Age=0 (clears cookie)
4. Frontend: clears Zustand auth state (access_token wiped from memory)

3.2 JWT Token Specification

Algorithm	HS256 (HMAC-SHA256). RS256 (asymmetric) to be evaluated for v1.5 multi-service setup.
Secret Key	Minimum 256-bit (32-byte) cryptographically random value. Generated with <code>secrets.token_hex(32)</code> . Stored in AWS Secrets Manager.
Access Token Expiry	1440 minutes (24 hours). Short enough to limit exposure window if stolen.
Refresh Token Expiry	30 days. Stored as a SHA-256 hashed UUID in Redis (not the raw token).
JWT Payload Fields	<code>{ sub: user_id (UUID), role: string, iat: unix_timestamp, exp: unix_timestamp }</code>
Payload Restrictions	No email, name, or any PII in JWT payload. Role is read from the verified token only — never from request body.
Token Storage (Client)	Access token: Zustand in-memory state only. Never localStorage (XSS risk). Never sessionStorage. Refresh token: httpOnly secure cookie.
Token Revocation	Refresh tokens are revocable via Redis deletion. Access tokens are not revocable (stateless by design) — 24h expiry limits the revocation gap.

3.3 Password Security Policy

Hashing Algorithm	bcrypt with work factor (cost) = 12. This results in ~250ms hash time per attempt — intentional to slow brute-force attacks.
Minimum Requirements	Minimum 8 characters. Must contain at least one uppercase letter, one lowercase letter, one number, and one special character.
Strength Validation	Frontend uses <code>zxcvbn</code> library to show real-time password strength meter. Backend enforces minimum requirements via <code>Pydantic</code> validator.
Storage Rules	Only the bcrypt hash is stored in the database. Raw password never touches the database, never appears in logs, never returned in any API response.
Failed Login Handling	After 5 consecutive failed login attempts for the same email, the account is temporarily locked for 15 minutes. Counter stored in Redis.
Password Reset	POST <code>/auth/forgot-password</code> sends a time-limited reset link (valid 1 hour) to the registered email. Link contains a signed one-time token.
Password Change	Requires current password verification before accepting new password. Invalidates all existing refresh tokens on change.

SECTION 4

Role-Based Access Control (RBAC)

4. Role-Based Access Control (RBAC)

4.1 Role Definitions

job_seeker	A registered candidate who can upload resumes, apply to jobs, view AI insights, take mock interviews, and track application status. Cannot post jobs or view other users data.
recruiter	A hiring professional who can post and manage job listings, view applicants for their own jobs only, update application statuses, and access hiring analytics. Cannot apply to jobs.
admin	A platform operator with full read/write access to all data for moderation purposes. Can deactivate users, remove jobs, and view platform-wide analytics. Requires additional 2FA.

4.2 Full Permission Matrix

Endpoint	job_seeker	recruiter	admin	Notes
POST /auth/register	Public	Public	Public	—
POST /auth/login	Public	Public	Public	—
GET /jobs (browse all)	ALLOWED	ALLOWED	ALLOWED	Any authenticated user
GET /jobs/:id (detail)	ALLOWED	ALLOWED	ALLOWED	Any authenticated user
POST /jobs (create)	BLOCKED 403	ALLOWED	ALLOWED	Recruiter must own to edit/delete
PUT /jobs/:id (update)	BLOCKED 403	Owner only	ALLOWED	Non-owner recruiter gets 403
DELETE /jobs/:id	BLOCKED 403	Owner only	ALLOWED	Non-owner recruiter gets 403
POST /applications (apply)	ALLOWED	BLOCKED 403	ALLOWED	Job seeker must have resume
GET /applications/me	ALLOWED	BLOCKED 403	ALLOWED	Own applications only
GET /applications/job/:id	BLOCKED 403	Owner only	ALLOWED	Recruiter must own that job

PATCH /applications/:id/status	BLOCKED 403	Job owner only	ALLOWED	Only shortlist/reject/hire
POST /resumes/upload	ALLOWED	BLOCKED 403	ALLOWED	One active resume per user
GET /resumes/me	ALLOWED	BLOCKED 403	ALLOWED	Own resume only
GET /users/:id (view candidate)	BLOCKED 403	ALLOWED	ALLOWED	Recruiter views applicants only
POST /ai/analyze-resume	ALLOWED	ALLOWED	ALLOWED	All authenticated users
POST /ai/score-candidate	ALLOWED	ALLOWED	ALLOWED	All authenticated users
POST /ai/recommend-jobs	ALLOWED	BLOCKED 403	ALLOWED	Job seekers only
POST /ai/evaluate-interview	ALLOWED	BLOCKED 403	ALLOWED	Job seekers only
GET /recruiter/analytics	BLOCKED 403	ALLOWED	ALLOWED	Own job analytics only
GET /recruiter/top-candidates	BLOCKED 403	ALLOWED	ALLOWED	Own jobs only
GET /admin/*	BLOCKED 403	BLOCKED 403	ALLOWED	Requires admin role + 2FA

4.3 RBAC Implementation

```
# FastAPI RBAC middleware - role_middleware.py

from functools import wraps
from fastapi import HTTPException, Depends
from app.core.security import get_current_user

def require_role(*allowed_roles: str):
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, current_user=Depends(get_current_user),
            **kwargs):
            if current_user.role not in allowed_roles:
                raise HTTPException(
                    status_code=403,
                    detail={
                        'error': 'Access denied',
                        'code': 'INSUFFICIENT_ROLE',
                        'required': list(allowed_roles),
                        'actual': current_user.role
                    }
                )
            return await func(*args, **kwargs)
        return wrapper
    return decorator
```

```
        return await func(*args, current_user=current_user, **kwargs)
    return wrapper
return decorator

# Usage on routes:
@router.post('/jobs')
@require_role('recruiter', 'admin')
async def create_job(job: JobCreate, current_user=Depends(get_current_user)):
    ...

# Ownership check (additional layer beyond role check):
async def verify_job_ownership(job_id: UUID, current_user: User, db:
AsyncSession):
    job = await db.get(Job, job_id)
    if not job:
        raise HTTPException(status_code=404, detail='Job not found')
    if job.recruiter_id != current_user.id and current_user.role != 'admin':
        raise HTTPException(status_code=403, detail='You do not own this job')
    return job
```

SECTION 5

Data Protection & Encryption

5. Data Protection & Encryption

5.1 Data Classification

Classification	Data Types	Handling Requirements
Top Secret	JWT secret keys, AWS root credentials, DB master password	Never stored in code. AWS Secrets Manager only. Access logged and alerted.
Confidential	User passwords (hashed), resume files, personal profile data	Encrypted at rest (AES-256). Access via authenticated APIs only. Logged.
Internal	Job postings, AI scores, application statuses, analytics	Stored in DB. Accessible to authorized roles. Paginated API access.
Public	Job titles, company names, job descriptions (published)	Accessible without authentication. Cached at CDN level.

5.2 Encryption Standards

Data in Transit	TLS 1.3 enforced on all connections (API, DB, Redis, S3). TLS 1.0 and 1.1 disabled at load balancer. Certificate pinning evaluated for v1.5.
Data at Rest (DB)	AWS RDS encryption using AES-256. Encryption key managed by AWS KMS. Key rotation every 365 days.
Data at Rest (S3)	Server-Side Encryption SSE-S3 (AES-256) on all objects. Bucket-level encryption policy enforces this — uploads without encryption are rejected.
Data at Rest (Redis)	ElastiCache in-transit encryption enabled. At-rest encryption enabled. Redis AUTH password required for all connections.
Passwords	bcrypt with cost factor 12. One-way hash — cannot be decrypted. Salted automatically by bcrypt algorithm.
Backup Encryption	RDS automated backups encrypted with the same KMS key as the primary database. Backups cannot be restored without the KMS key.

5.3 AWS S3 Security Configuration

```
# S3 Bucket Policy – deny all public access
{
  'Version': '2012-10-17',
  'Statement': [
    {
      'Sid': 'DenyPublicAccess',
      'Effect': 'Deny',
      'Principal': '*',
      'Action': 's3:GetObject',
      'Resource': 'arn:aws:s3:::hireiq-resumes-prod/*',
      'Condition': {
        'StringNotEquals': {
          'aws:PrincipalArn': 'arn:aws:iam::ACCOUNT_ID:role/hireiq-api-role'
        }
      }
    }
  ]
}

# Pre-signed URL generation (Python / boto3):
def generate_upload_url(user_id: str, filename: str) -> str:
    key = f'resumes/{user_id}/{uuid4()}.pdf'
    url = s3_client.generate_presigned_url(
        'put_object',
        Params={
            'Bucket': settings.S3_BUCKET_NAME,
            'Key': key,
            'ContentType': 'application/pdf',
            'ContentLength': MAX_FILE_SIZE, # 5MB enforcement
        },
        ExpiresIn=900 # 15 minutes
    )
    return url, key
```

5.4 Sensitive Data Handling Rules

Never Log	Passwords (raw or hashed), JWT secret key, AWS credentials, S3 pre-signed URLs, full credit card numbers (if added in future).
Never Return in API	password_hash field, JWT secret, internal user IDs of other users (use UUIDs only), Redis keys, internal service URLs.
PII Minimization	Collect only required PII: name, email, skills, experience. No government ID, no date of birth, no marital status collected.
Data Retention	User data retained as long as account is active. Deleted within 30 days of account deletion request (GDPR Article 17). Resume files deleted from S3 within 24 hours.
Log Masking	Email addresses in logs are masked: john@example.com → j***@example.com. User IDs use UUID format (not sequential integers).

SECTION 6

OWASP Top 10 Compliance Checklist

6. OWASP Top 10 (2021) Compliance Checklist

All items in this checklist must be marked PASS before the platform is approved for production release. Items marked IN PROGRESS must have a completion date assigned.

OWASP Category	Implementation Status & Notes	Status	Deadline
A01 — Broken Access Control	Implemented RBAC on all endpoints. Ownership checks for jobs and applications. No direct object references (UUIDs not sequential IDs). Tests written.	PASS	Pre-launch
A02 — Cryptographic Failures	TLS 1.3 on all connections. bcrypt for passwords. AES-256 for data at rest. No MD5/SHA1. Secrets in AWS Secrets Manager.	PASS	Pre-launch
A03 — Injection	SQLAlchemy ORM parameterized queries (no raw SQL). Pydantic input validation. No shell commands with user input. spaCy NLP sandboxed.	PASS	Pre-launch
A04 — Insecure Design	Threat model (STRIDE) completed. Secure-by-default: auth required on all routes except explicit public list. Security review in design phase.	PASS	Pre-launch
A05 — Security Misconfiguration	S3 public access block enabled. Debug mode off in production. No default credentials. Security headers on all responses. Dockerfile uses non-root user.	IN PROGRESS	Week 12
A06 — Vulnerable Components	requirements.txt and package.json pinned to specific versions. Dependabot enabled on GitHub. Weekly automated vulnerability scan via Safety (Python) and npm audit.	IN PROGRESS	Week 11
A07 — Auth & Session Failures	JWT with 24h expiry. Refresh token rotation. bcrypt password hashing. Account lockout after 5 failed attempts. No credentials in URLs.	PASS	Pre-launch
A08 — Software & Data Integrity	GitHub branch protection: main requires PR review + CI passing. Docker images signed (Cosign). No auto-update of unverified packages.	IN PROGRESS	Week 12
A09 — Logging & Monitoring	Structured JSON logs on all services. CloudWatch alerting. All auth events logged. Audit trail for sensitive actions. Log retention 30 days.	PASS	Pre-launch
A10 — SSRF	AI service not publicly exposed (internal VPC only). No user-controlled URLs fetched by the server. URL validation on any external link inputs.	PASS	Pre-launch

6.1 Security Headers Configuration

```
# FastAPI security headers middleware
# Applied to ALL responses via middleware

@app.middleware('http')
async def add_security_headers(request: Request, call_next):
    response = await call_next(request)
    response.headers['Strict-Transport-Security'] = 'max-age=31536000;
includeSubDomains'
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['X-XSS-Protection'] = '1; mode=block'
    response.headers['Referrer-Policy'] = 'strict-origin-when-cross-
origin'
    response.headers['Content-Security-Policy'] = (
        default-src self;
        script-src self unsafe-inline;
        style-src self unsafe-inline;
        img-src self data: https://cdn.hireiq.io;
        connect-src self https://api.hireiq.io
    )
    response.headers['Permissions-Policy'] = 'camera=(), microphone=(),
geolocation=()'
    del response.headers['Server'] # Remove server version info
    return response
```

SECTION 7

Input Validation & Injection Prevention

7. Input Validation & Injection Prevention

7.1 Validation Layers

HireIQ applies input validation at three distinct layers to ensure no malicious input reaches the database or AI engine:

Layer 1: Frontend (React)	Zod schema validation on all form inputs before submission. React Hook Form enforces field constraints. File type and size checked in the browser before upload.
Layer 2: API Gateway / Middleware	Rate limiting middleware blocks excessive requests before they reach application logic. Request size limits enforced at the load balancer level (max 10MB body).
Layer 3: FastAPI / Pydantic	All request bodies validated against Pydantic v2 schemas with strict mode. Invalid fields return HTTP 422 with field-level error details. Extra fields are rejected.

7.2 Pydantic Schema Examples

```
# schemas/job.py - Job creation schema with strict validation
from pydantic import BaseModel, Field, field_validator
from typing import List

class JobCreate(BaseModel):
    model_config = ConfigDict(strict=True, str_strip_whitespace=True)

    title: str = Field(min_length=3, max_length=255)
    description: str = Field(min_length=50, max_length=10000)
    required_skills: List[str] = Field(min_length=1, max_length=20)
    salary_min: int = Field(ge=0, le=10000000)
    salary_max: int = Field(ge=0, le=10000000)
    location: str = Field(max_length=255)
    job_type: Literal['full_time', 'part_time', 'contract', 'remote']

    @field_validator('required_skills')
    @classmethod
    def validate_skills(cls, v):
        for skill in v:
            if len(skill) > 50 or not skill.replace(' ', '').isalnum():
                raise ValueError(f'Invalid skill: {skill}')
        return [s.strip().lower() for s in v]
```

```

@field_validator('salary_max')
@classmethod
def validate_salary_range(cls, v, values):
    if v < values.data.get('salary_min', 0):
        raise ValueError('salary_max must be >= salary_min')
    return v

# schemas/resume.py - File upload validation
ALLOWED_TYPES = {'application/pdf'}
MAX_SIZE_BYTES = 5 * 1024 * 1024 # 5MB

async def validate_resume_file(file: UploadFile) -> UploadFile:
    if file.content_type not in ALLOWED_TYPES:
        raise HTTPException(400, detail='Only PDF files are accepted')
    content = await file.read(MAX_SIZE_BYTES + 1)
    if len(content) > MAX_SIZE_BYTES:
        raise HTTPException(400, detail='File size exceeds 5MB limit')
    await file.seek(0) # Reset for downstream processing
    return file

```

7.3 SQL Injection Prevention

ORM-Only Policy	All database interactions use SQLAlchemy ORM or Core with bound parameters. Raw SQL string interpolation is prohibited by code review policy.
Parameterized Queries	SQLAlchemy automatically parameterizes all queries. User input is never concatenated into SQL strings at any point in the codebase.
No Dynamic Table Names	Table names and column names are never constructed from user input. All schema references are hardcoded in ORM model definitions.
DB User Permissions	The application DB user has only SELECT, INSERT, UPDATE, DELETE on application tables. No DROP, CREATE, TRUNCATE, or ALTER permissions.
Automated Scanning	SQLMap-based testing runs in CI pipeline on staging environment before every production release.

7.4 XSS Prevention

React Auto-Escaping	React escapes all dynamic content inserted into JSX by default. No use of dangerouslySetInnerHTML anywhere in the codebase.
Content Security Policy	CSP header restricts script sources to self only. Inline scripts blocked except for Next.js runtime requirements. Evaluated strictly.
Output Encoding	All user-generated content (job descriptions, bio, skills) rendered via React — never injected as raw HTML. Markdown rendering uses sanitized parser.

API Response

FastAPI JSON responses automatically escape special HTML characters in string fields. No raw HTML returned in API responses.

SECTION 8

Rate Limiting & DDoS Protection

8. Rate Limiting & DDoS Protection

8.1 Rate Limiting Configuration

Endpoint	Limit	Action	Rationale
POST /auth/register	10 req/min per IP	Block 429	Redis sliding window counter. Prevents bulk account creation.
POST /auth/login	10 req/min per IP	Block 429 + account lock after 5 fails	Prevents brute-force password attacks.
POST /auth/refresh	30 req/min per IP	Block 429	Prevents refresh token flooding.
GET /jobs (listing)	200 req/min per user	Throttle 429	Prevents scraping of entire job database.
POST /jobs (create)	20 req/min per user	Block 429	Prevents job spam flooding the platform.
POST /resumes/upload	5 req/min per user	Block 429	Prevents storage abuse and cost amplification attacks.
POST /ai/analyze-resume	10 req/min per user	Block 429	Prevents AI resource exhaustion attacks.
POST /ai/score-candidate	20 req/min per user	Block 429	AI inference is CPU-intensive — limits abuse.
POST /applications	30 req/min per user	Block 429	Prevents application spam across all jobs.
General API (all others)	1000 req/min per IP	Throttle 429	Catch-all rate limit for unanticipated patterns.

8.2 Rate Limiter Implementation

```
# middlewares/rate_limit.py - Redis sliding window rate limiter
import time
from fastapi import Request, HTTPException
from app.core.redis import redis_client

async def rate_limit(request: Request, max_requests: int, window_seconds: int):
```

```

# Use IP + endpoint as the key for granular limiting
client_ip = request.client.host
endpoint = request.url.path
key = f'rate:{client_ip}:{endpoint}'
now = int(time.time())
window_start = now - window_seconds

async with redis_client.pipeline(transaction=True) as pipe:
    # Sliding window: remove expired entries, add current timestamp
    await pipe.zremrangebyscore(key, 0, window_start)
    await pipe.zadd(key, {str(now): now})
    await pipe.zcard(key) # Count requests in window
    await pipe.expire(key, window_seconds)
    results = await pipe.execute()

request_count = results[2]
if request_count > max_requests:
    raise HTTPException(
        status_code=429,
        detail={
            'error': 'Rate limit exceeded',
            'code': 'RATE_LIMIT_EXCEEDED',
            'retry_after': window_seconds
        },
        headers={'Retry-After': str(window_seconds)}
    )

```

8.3 DDoS Protection Layers

Layer 1: AWS CloudFront	CloudFront absorbs volumetric DDoS attacks at the edge before traffic reaches the origin. AWS Shield Standard included at no extra cost.
Layer 2: AWS WAF	Web Application Firewall rules block: SQL injection patterns, XSS patterns, known bad IPs, requests from sanctioned countries (configurable).
Layer 3: ALB Connection Limits	Application Load Balancer enforces maximum connection limits and request timeouts. Slowloris attacks mitigated by ALB idle timeout (60s).
Layer 4: Rate Limiting	Redis-backed per-IP and per-user rate limiting as defined in Section 8.1. Applied at application level as a final layer.
Layer 5: ECS Auto-Scaling	ECS Fargate auto-scales on CPU/memory pressure. Even under sustained load, capacity increases to maintain service availability.

SECTION 9

Audit Logging & Monitoring

9. Audit Logging & Security Monitoring

9.1 Mandatory Audit Events

The following events MUST generate an audit log entry regardless of success or failure:

Event Code	Event	Fields Logged
AUTH-001	User registration (success or failure)	user_id (if created), email_masked, ip_address, user_agent, timestamp
AUTH-002	User login (success or failure)	email_masked, success: bool, ip_address, user_agent, timestamp, failure_reason
AUTH-003	Token refresh	user_id, ip_address, timestamp, new_token_expiry
AUTH-004	User logout	user_id, ip_address, timestamp
AUTH-005	Account lockout (5 failed logins)	email_masked, ip_address, lockout_until, timestamp
AUTH-006	Password change	user_id, ip_address, timestamp — NOT old or new password
JOB-001	Job created	job_id, recruiter_id, title, timestamp
JOB-002	Job updated	job_id, recruiter_id, changed_fields[], timestamp
JOB-003	Job deleted	job_id, recruiter_id, timestamp
JOB-004	Job closed	job_id, recruiter_id, timestamp
APP-001	Application submitted	application_id, user_id, job_id, resume_url, timestamp
APP-002	Application status changed	application_id, recruiter_id, old_status, new_status, timestamp
RES-001	Resume uploaded	user_id, file_size_bytes, s3_key, timestamp
RES-002	Resume deleted	user_id, s3_key, timestamp
SEC-001	Rate limit exceeded	ip_address, endpoint, request_count, timestamp
SEC-002	Unauthorized access attempt (401/403)	user_id (if any), endpoint, ip_address, timestamp
SEC-003	Invalid JWT token received	ip_address, endpoint, token_prefix, timestamp

ADMIN-001	Admin action performed	admin_user_id, action, target_id, details, timestamp
------------------	------------------------	--

9.2 Audit Log Format

```
# Example audit log entry (structured JSON)
{
  'timestamp': '2025-06-14T09:30:00.123Z',
  'level': 'INFO',
  'event_code': 'APP-002',
  'event': 'application_status_changed',
  'request_id': 'req_8f3a2b1c-d4e5-...',
  'actor': {
    'user_id': 'usr_7c4d9a2e-...',
    'role': 'recruiter',
    'ip': '203.0.113.42',
    'user_agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X ...)'
  },
  'resource': {
    'type': 'application',
    'id': 'app_1b2c3d4e-...',
    'job_id': 'job_9f8e7d6c-...'
  },
  'change': {
    'field': 'status',
    'old_value': 'applied',
    'new_value': 'shortlisted'
  },
  'outcome': 'success'
}
```

9.3 Security Alerting Rules

Trigger Condition	Severity	Response Action
5+ failed logins on same account within 5 min	High	Trigger account lockout + notify user via email + log SEC event
100+ requests/min from single IP on /auth/*	High	Block IP for 1 hour + log SEC-001 + alert on-call engineer
Any SEC-003 invalid JWT event	Medium	Log and monitor. Alert if > 50 events in 10 min from same IP.
Any SEC-002 403 event from authenticated user	Medium	Log all. Alert if > 20 per user in 10 min (potential enumeration)
Admin action (ADMIN-001) outside business hours	High	Immediate alert to security team via PagerDuty
S3 bucket policy change detected	Critical	Immediate alert. Auto-revert via AWS Config rule if possible.

New IAM key created or permissions changed	Critical	Immediate alert to security team. Require approval within 1 hour.
DB CPU > 80% sustained for 10 minutes	High	Alert to on-call. Check for long-running queries or injection.

SECTION 10

GDPR & Data Privacy Compliance

10. GDPR & Data Privacy Compliance

HireIQ processes personal data of users in the EU and worldwide. Full GDPR compliance is required before the platform launches publicly. The following section defines the obligations and implementation approach.

10.1 Legal Basis for Processing

User Registration & Profile	Contractual necessity — processing required to provide the service the user signed up for (Article 6(1)(b))
Resume Processing & AI Analysis	Explicit consent — user actively uploads resume and consents to AI analysis via in-app consent prompt (Article 6(1)(a))
Application History	Contractual necessity — job application records required for the service to function (Article 6(1)(b))
Analytics & Aggregated Insights	Legitimate interest — aggregated, anonymized data used to improve the platform (Article 6(1)(f))
Email Communications	Consent — user opts in to email notifications. Opt-out available at any time (Article 6(1)(a))

10.2 Data Subject Rights Implementation

Right to Access (Art. 15)	GET /users/me/data-export — returns all personal data in JSON format within 30 days. Includes profile, applications, resume metadata, AI scores.
Right to Erasure (Art. 17)	DELETE /users/me/account — permanently deletes all personal data within 30 days. Resume deleted from S3 within 24 hours. Anonymized aggregate data may be retained.
Right to Rectification (Art. 16)	PUT /users/profile — user can update all personal data fields at any time through the profile edit page.
Right to Portability (Art. 20)	GET /users/me/data-export returns data in machine-readable JSON format. CSV export option for applications and profile data.
Right to Object (Art. 21)	Users can opt out of AI profiling via account settings. Opting out disables AI scoring and recommendations but allows manual job browsing.
Right to Restrict Processing (Art. 18)	Account deactivation option available. Deactivated accounts retain data but stop all AI processing. Data deleted if not reactivated within 90 days.

10.3 Consent Management

Registration Consent	On registration, users must explicitly check: (1) Terms of Service, (2) Privacy Policy, (3) AI Analysis consent. Pre-checked boxes are not used.
AI Analysis Consent	A separate explicit consent is required for AI resume analysis. Users can use the platform without AI features by declining this consent.
Cookie Consent	Cookie banner on first visit. Only strictly necessary cookies (auth) active by default. Analytics cookies require opt-in.
Consent Records	All consent events stored in a consent_log table with: user_id, consent_type, granted: bool, ip_address, timestamp, version of policy accepted.
Withdrawal	Users can withdraw any consent from Account Settings > Privacy. Withdrawal is effective immediately. Past processing remains lawful.

10.4 Data Breach Response Procedure

Detection	Automated: CloudWatch alarm on anomalous data export patterns. Manual: user report or penetration test finding.
Containment (0-4 hrs)	Immediately revoke compromised credentials. Block affected IP ranges. Isolate affected service. Preserve logs for forensic investigation.
Assessment (4-24 hrs)	Determine: scope of breach, data types affected, number of users impacted, whether breach is ongoing, entry vector.
Notification — Supervisory Authority	GDPR Article 33: notify the relevant Data Protection Authority within 72 hours of becoming aware of the breach.
Notification — Users (if required)	GDPR Article 34: notify affected users without undue delay if the breach is likely to result in high risk to their rights and freedoms.
Remediation	Fix the vulnerability that allowed the breach. Force password reset for affected users. Review and update security controls.
Post-Incident Report	Document the breach, timeline, impact, response actions, and lessons learned. Report to leadership within 7 days.

SECTION 11

Penetration Testing Checklist

11. Pre-Launch Penetration Testing Checklist

The following penetration testing checklist must be completed by an independent security reviewer before the production launch is approved. All findings rated High or Critical must be resolved before go-live.

11.1 Authentication Testing

Test ID	Test Description	Priority
PENTEST-AUTH-01	Test registration with missing required fields — verify 422 response with field details	REQUIRED
PENTEST-AUTH-02	Test login with wrong password — verify 401 with generic message (no field-level hint)	REQUIRED
PENTEST-AUTH-03	Test login brute-force: 10+ attempts — verify account lockout triggers	REQUIRED
PENTEST-AUTH-04	Test JWT token replay after logout — verify 401 response (refresh token revoked)	REQUIRED
PENTEST-AUTH-05	Test expired access token — verify 401 and automatic refresh flow works	REQUIRED
PENTEST-AUTH-06	Test tampered JWT payload (change role field) — verify signature validation rejects it	REQUIRED
PENTEST-AUTH-07	Test API call with no Authorization header on protected route — verify 401	REQUIRED
PENTEST-AUTH-08	Test concurrent login from 2 devices — verify both sessions work independently	REQUIRED

11.2 Authorization Testing

Test ID	Test Description	Priority
PENTEST-RBAC-01	Job seeker attempts POST /jobs — verify 403 Forbidden	REQUIRED
PENTEST-RBAC-02	Recruiter attempts POST /applications — verify 403 Forbidden	REQUIRED

PENTEST-RBAC-03	Recruiter A attempts to update Recruiter B's job — verify 403 with ownership check error	REQUIRED
PENTEST-RBAC-04	Job seeker attempts GET /applications/job/:id (recruiter endpoint) — verify 403	REQUIRED
PENTEST-RBAC-05	Any user attempts GET /admin/* without admin role — verify 403	REQUIRED
PENTEST-RBAC-06	Job seeker attempts to view another user's resume via GET /resumes/:other_user_id — verify 403	REQUIRED
PENTEST-RBAC-07	IDOR test: user attempts PATCH /applications/:id/status with another user's application ID	REQUIRED

11.3 Injection & Input Testing

Test ID	Test Description	Priority
PENTEST-INJ-01	SQL injection in job search: search=' OR 1=1-- — verify safe parameterized response	REQUIRED
PENTEST-INJ-02	XSS in job title: <script>alert(1)</script> — verify output is escaped, not executed	REQUIRED
PENTEST-INJ-03	Large payload injection: 10MB JSON body — verify request size limit rejects it	REQUIRED
PENTEST-INJ-04	Upload a non-PDF file (e.g. .exe) with content-type: application/pdf — verify rejection	REQUIRED
PENTEST-INJ-05	Upload PDF larger than 5MB — verify file size rejection before S3 upload	REQUIRED
PENTEST-INJ-06	Path traversal in job ID: GET /jobs/../../../../etc/passwd — verify 404 or 400	REQUIRED
PENTEST-INJ-07	NoSQL-style injection in skills filter: skills[]=\$where:function() — verify safe handling	RECOMMENDED

11.4 Transport & Cryptography Testing

Test ID	Test Description	Priority
PENTEST-CRYPT-01	Verify TLS 1.0 and 1.1 are disabled — only TLS 1.2+ accepted	REQUIRED
PENTEST-CRYPT-02	Verify all API responses include HSTS header with max-age >= 31536000	REQUIRED
PENTEST-CRYPT-03	Verify no credentials appear in URL query parameters or server logs	REQUIRED

PENTEST- CRYPT-04	Verify S3 pre-signed URL expires within 15 minutes — attempt access after expiry	REQUIRED
PENTEST- CRYPT-05	Verify S3 bucket is not publicly accessible without a pre-signed URL	REQUIRED
PENTEST- CRYPT-06	Verify API responses do not return password_hash field in any user response	REQUIRED

SECTION 12

Incident Response & Key Rotation

12. Incident Response & Secret Rotation

12.1 Incident Severity Classification

Severity	Description	Response SLA & Team
P0 — Critical	Active data breach, production database compromised, all user data exposed	Immediate response. CTO + security lead paged. All hands. < 1 hour SLA.
P1 — High	Auth bypass discovered, S3 bucket misconfiguration, mass account takeover	Response within 4 hours. Security team + on-call engineer. Service may be taken offline.
P2 — Medium	Rate limiting bypass, minor information disclosure, single account compromise	Response within 24 hours. Security team review and patch in next sprint.
P3 — Low	Non-sensitive info in error messages, outdated dependency with low CVSS	Log and schedule fix in next sprint. No immediate action required.

12.2 Incident Response Runbook

Step 1 — Detect	Alert fires in CloudWatch / Datadog OR user reports suspicious activity OR penetration tester finds vulnerability.
Step 2 — Triage	On-call engineer assesses severity (P0–P3). P0/P1: immediately page CTO and security lead via PagerDuty.
Step 3 — Contain	Revoke compromised credentials. Block attacker IP. Isolate affected service. Enable maintenance mode if needed to protect user data.
Step 4 — Investigate	Pull CloudWatch logs. Identify entry vector, scope, data accessed, timeline. Preserve all logs — do not delete or overwrite.
Step 5 — Eradicate	Patch the vulnerability. Rotate all potentially compromised secrets. Deploy fix to staging. Validate fix with security test.
Step 6 — Recover	Deploy patched code to production. Restore service. Force password reset for affected users. Monitor for 24 hours post-recovery.
Step 7 — Report	Write post-mortem: timeline, root cause, impact, actions taken. Share with leadership within 7 days. Notify DPA if required (GDPR).

12.3 Secret Rotation Schedule

Secret	Rotation Frequency	Rotation Procedure
JWT_SECRET_KEY	Every 90 days	Rotate in AWS Secrets Manager. Deploy new key to all ECS tasks. Short token expiry (24h) limits transition disruption.
Database Master Password	Every 180 days	Rotate via AWS Secrets Manager. RDS supports secret rotation without downtime using two-phase credential update.
AWS IAM Access Keys	Every 90 days	Use IAM roles for ECS tasks (no static keys). Service account keys for CI rotate every 90 days via GitHub Actions secrets.
S3 Pre-Signed URL Keys	Per request	Each URL is generated fresh per request with 15-min expiry. No key rotation needed — access is key-derived per request.
Redis AUTH Password	Every 180 days	Rotate in ElastiCache. Update ECS environment variables. Restart ECS tasks for graceful pickup.
Third-Party API Keys	On provider schedule	Any external service keys (email, Stripe in future) rotated per provider recommendation, minimum annually.

HireIQ SACD v1.0 — End of Document

This Security & Access Control Document governs all security policies, threat mitigation, and compliance requirements for HireIQ v1.0.